Vector Semantics

Natural Language Processing

Ayesha Enayet

ANN review

- Input
- Feed-forward
- Error
- Gradient
- Using some optimization algorithm we update weights

Word semantics (Lexical Semantics)

- Synonyms: two words are synonymous if they are substitutable one for the other in any sentence without changing the truth conditions of the sentence.
- Antonyms are words with an opposite meaning.

Different view

- Relatedness/association:
  - Coffee and Cup: they are associated in the world by commonly co-participating in a shared event.

Word Semantics (redefined)

- Instead of using some logical language to define each word, we should define words by some representation of how the word was used by actual people in speaking and understanding.
- Define a word by the environment or distribution it occurs in in language use (Joos (1950), Harris (1954), and Firth (1957)).
- A **word's distribution** is the set of contexts in which it occurs, the neighboring words or grammatical environments.
- The idea is that two words that occur in very similar distributions (that occur together with very similar words) are likely to have the same meaning.

Semantic field

- One common kind of relatedness between words is if they belong to the same semantic field semantic field.
- A semantic field is a set of words which cover a particular semantic domain and bear structured relations with each other.

- A word (or sense) is a **hyponym** of another word or sense if the first is more specific, denoting a subclass of the other.
  - For example, car is a hyponym of vehicle.
  - Vehicle is a **hypernym** of car,

Connotation

- The word connotation has different meanings in different fields, but here we use it to mean the aspects of a word's meaning that are related to a writer or reader's emotions, sentiment, opinions, or evaluations.

Concept of distribution

- A word's distribution is the set of contexts in which it occurs, the neighboring words or grammatical environments. The idea is that two words that occur in very similar distributions (that occur together with very similar words) are likely to have the same meaning.

Vector semantics

- The idea of vector semantics is thus to represent a word as a point in some multidimensional semantic space. Vectors for representing words are generally called embeddings , because the word is embedded in a particular vector space.

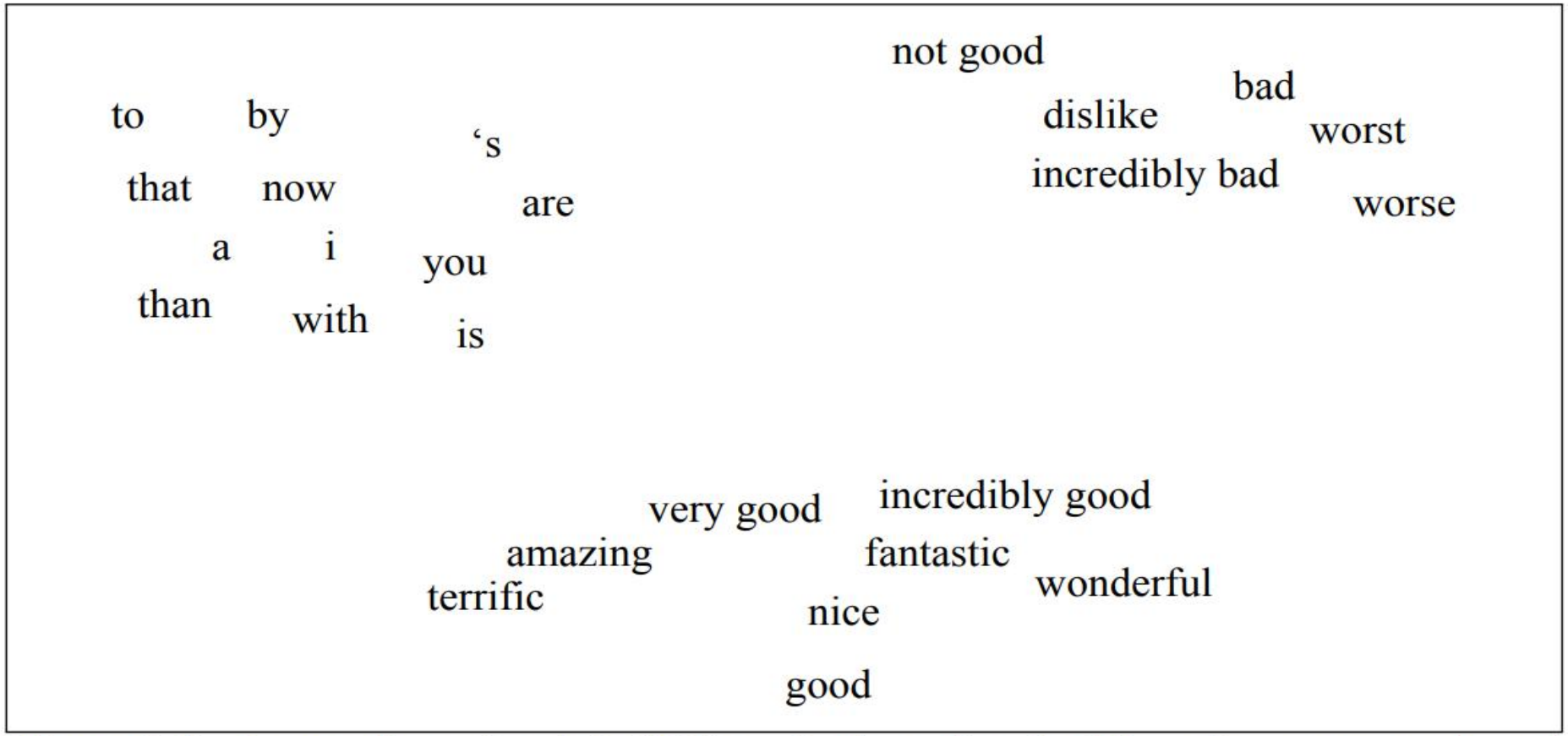


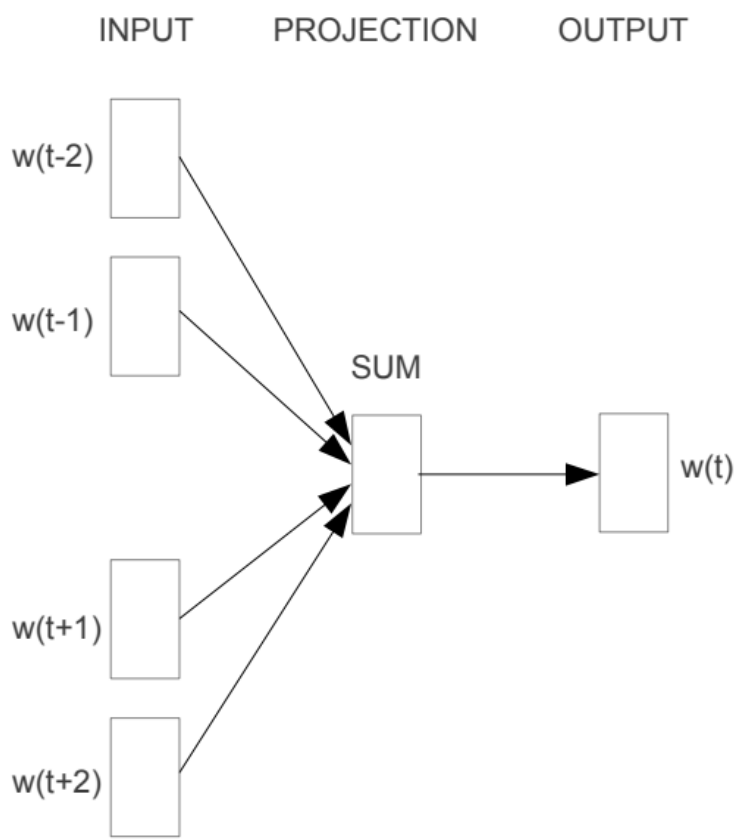
Figure 6.1 A two-dimensional (t-SNE) projection of embeddings for some words and phrases, showing that words with similar meanings are nearby in space. The original 60-dimensional embeddings were trained for a sentiment analysis task. Simplified from Li et al. (2015).

Word2Vec

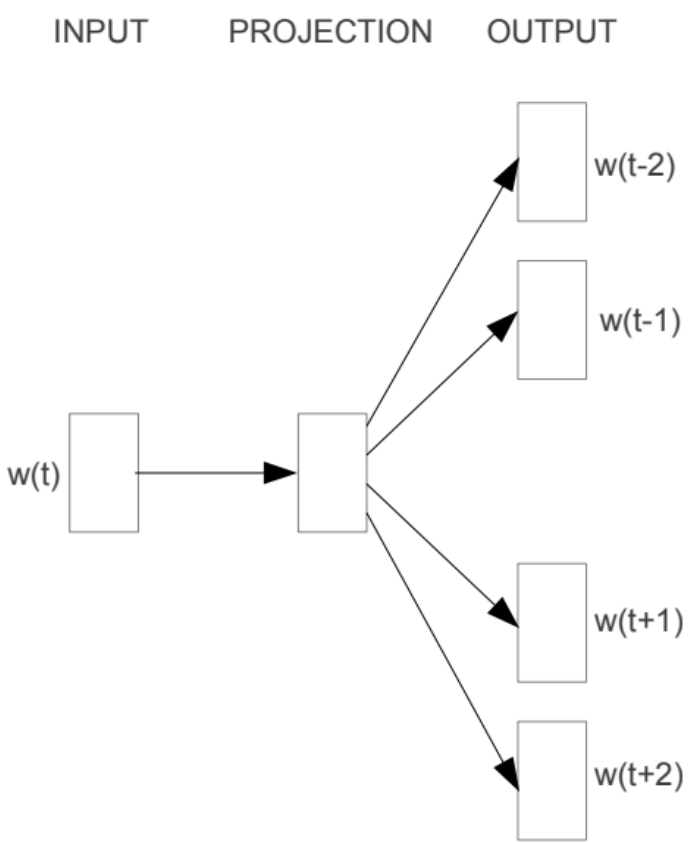
Instructor: Ayesha Enayet

Word2Vec

- We can train a classifier to predict: “Is word w likely to show up near word w_i or vice versa?”
- We don’t actually care about this prediction task; instead we’ll take the learned classifier weights as the word embeddings.



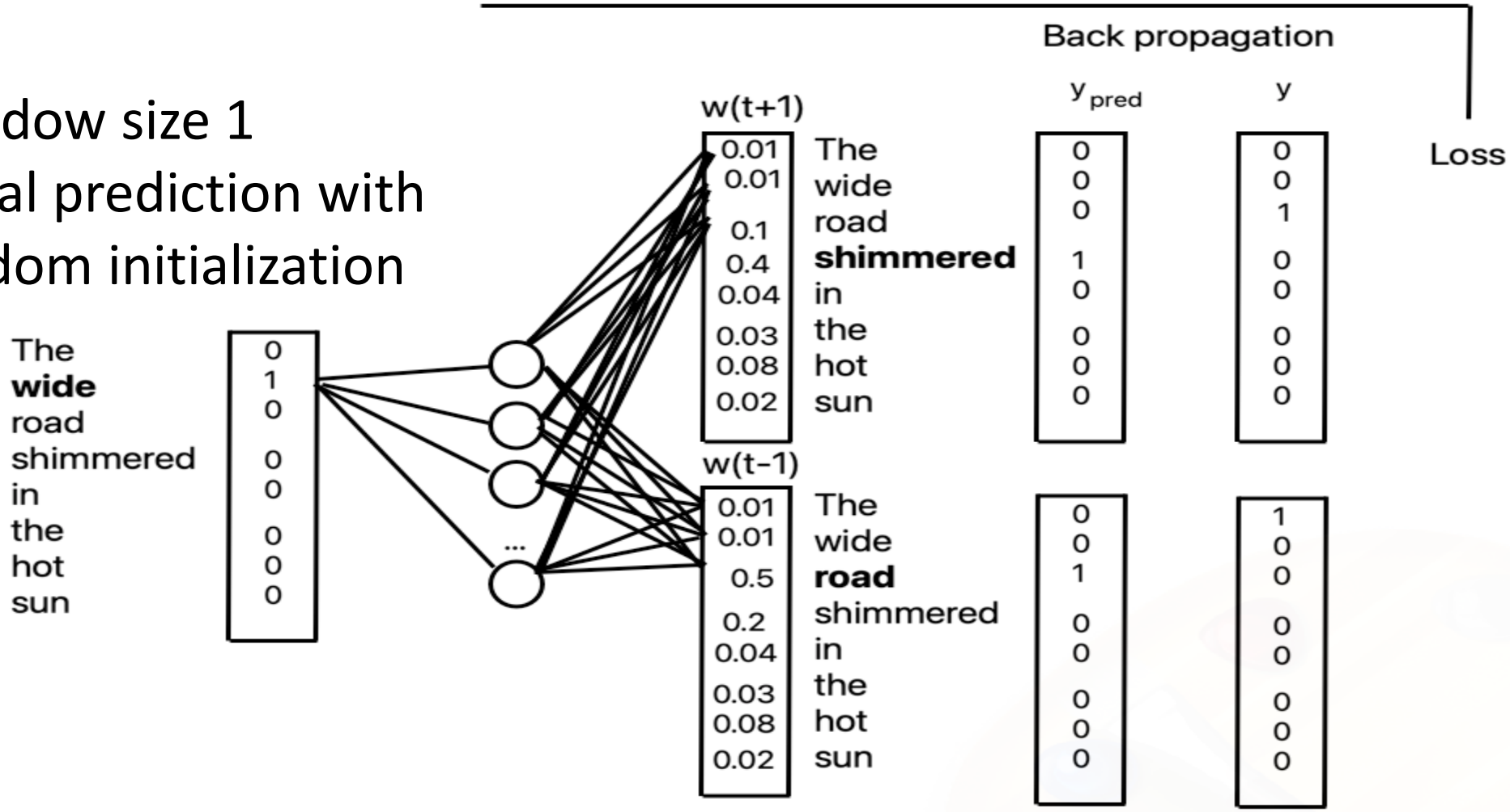
CBOW



Skip-gram

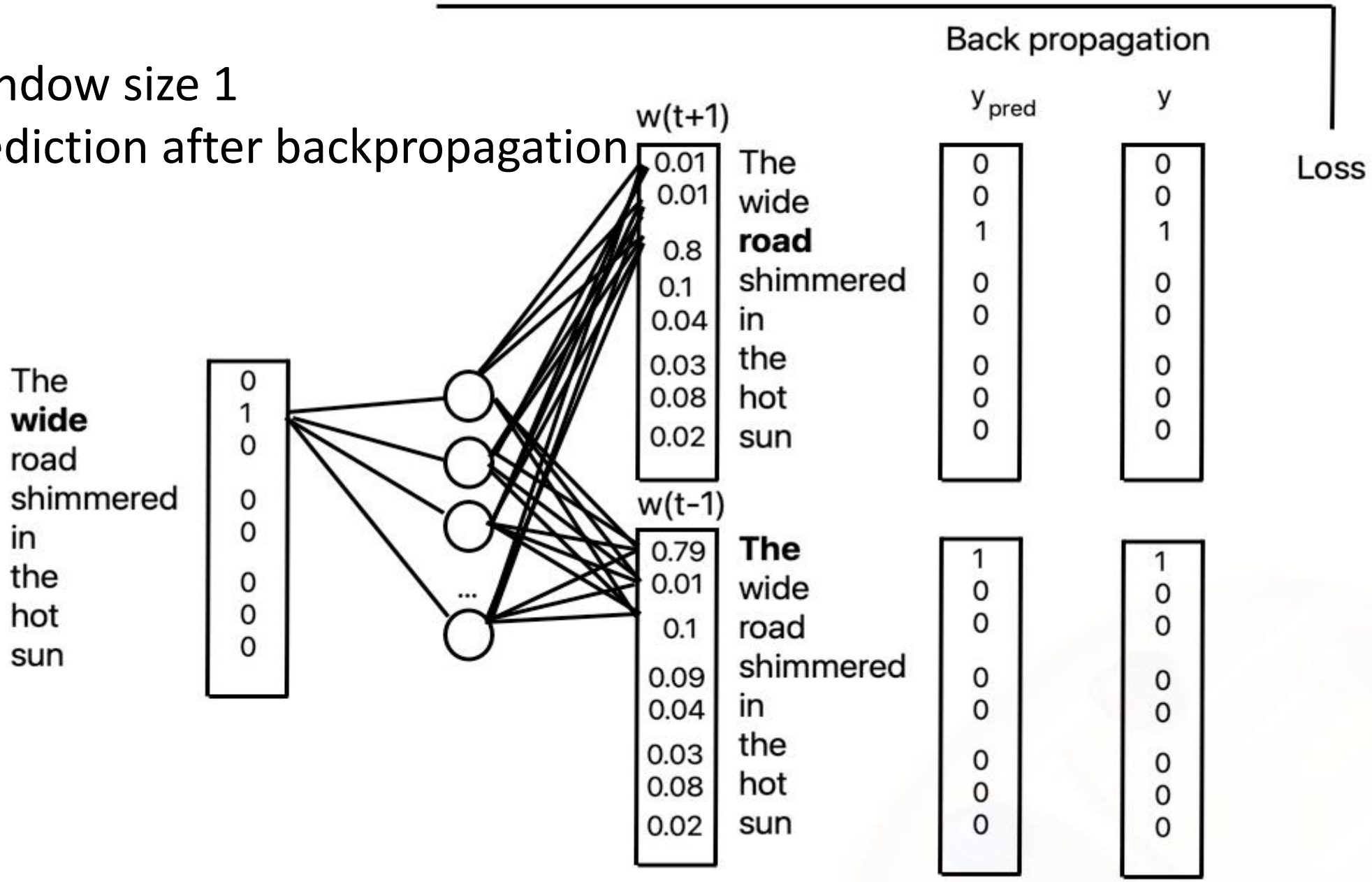
https://medium.com/@corymaklin/word2vec-skip-gram-904775613b4c

Window size 1
Initial prediction with random initialization



Window size 1

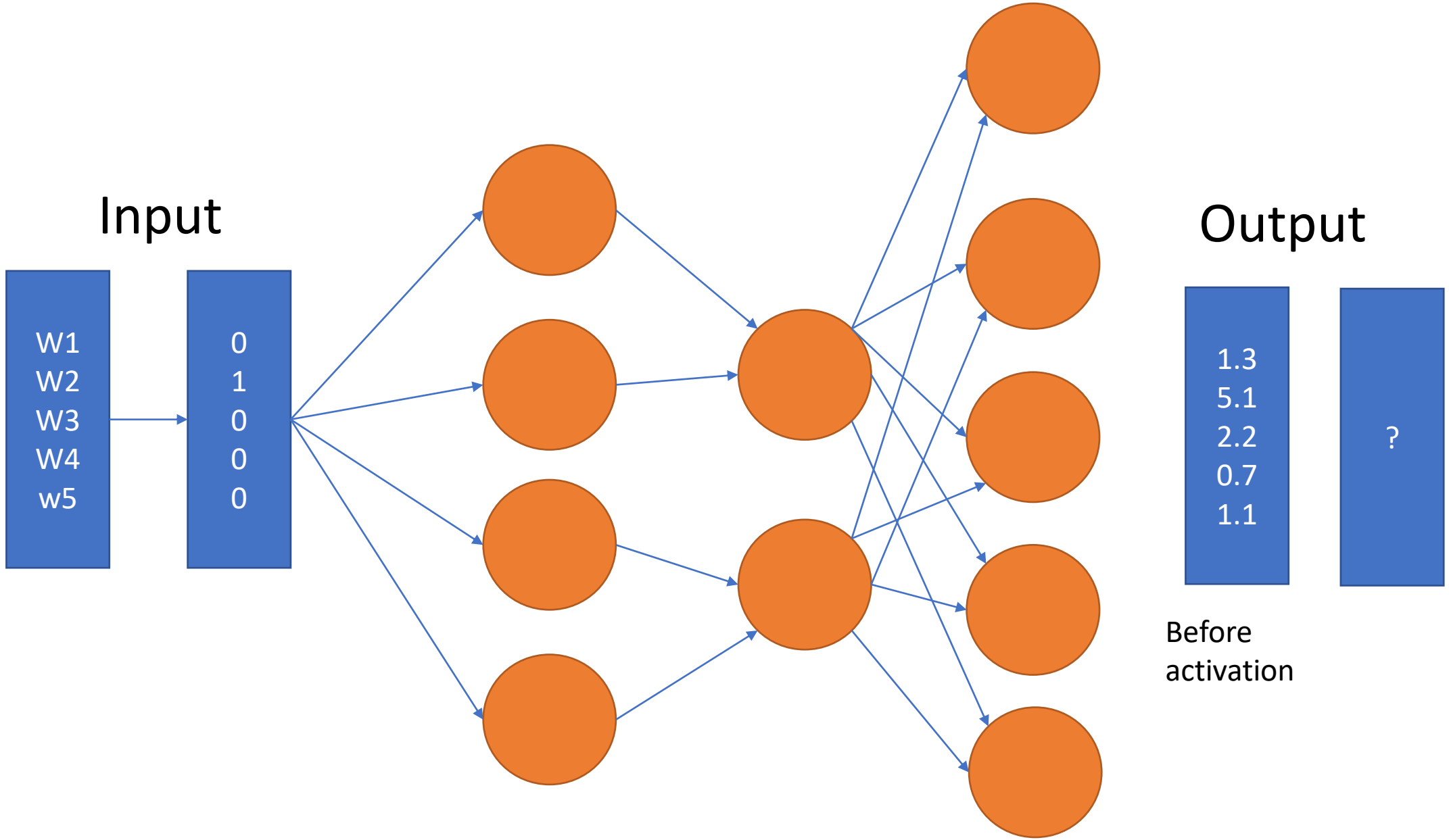
Prediction after backpropagation



Using Softmax activation multi-class classification

Example (Single input at a time)

- "The earth revolves around the sun. The moon revolves around the earth"
- Vocabulary=[‘around’, ’earth’, ’moon’, ’revolves’, ’sun’, ’the’]
- Input dimension= 1 X V= 1 X 6
- W1 dimension =6 X 10
- W2 dimension= 10 X 6
- Output dimension 1 X 6


Input
W1
W2
W3
W4
w5
0
1
0
0
0
Output
1.3
5.1
2.2
0.7
1.1
?
Before
activation

Softmax activation function

$$softmax(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{N} e^{z_j}}$$

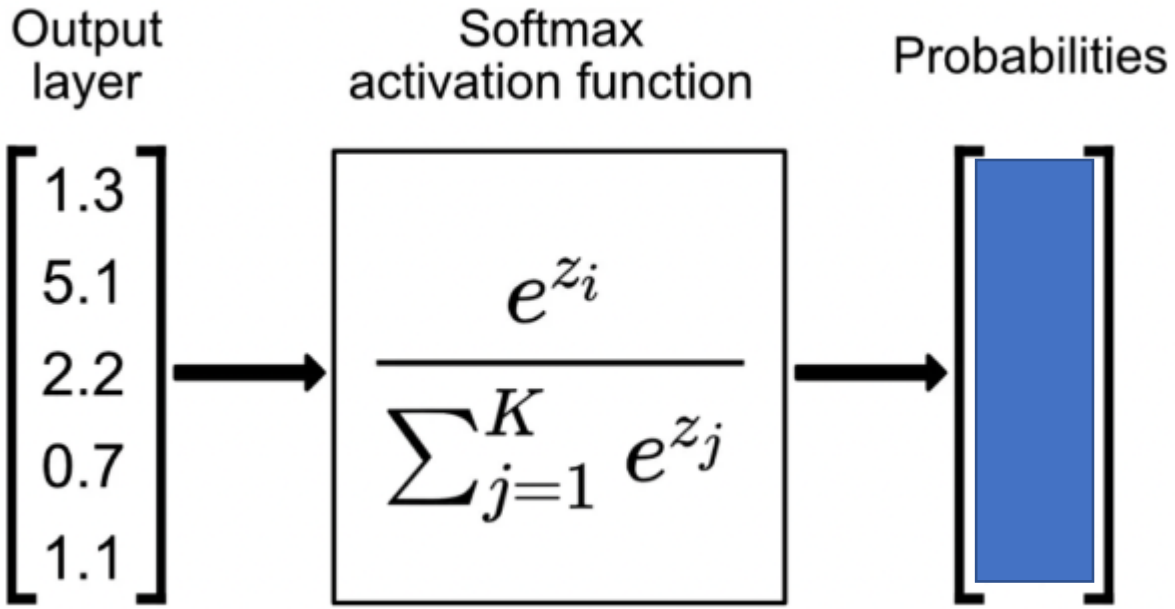
N is # of classes

Here,

- **z** is the vector of raw outputs from the neural network
- The value of e ≈ 2.718
- The i-th entry in the softmax output vector softmax(**z**) can be thought of as the predicted probability of the test input belonging to class i.

https://www.pinecone.io/learn/softmax-activation/

Softmax Calculation



$e^{1.3} = 3.669297$

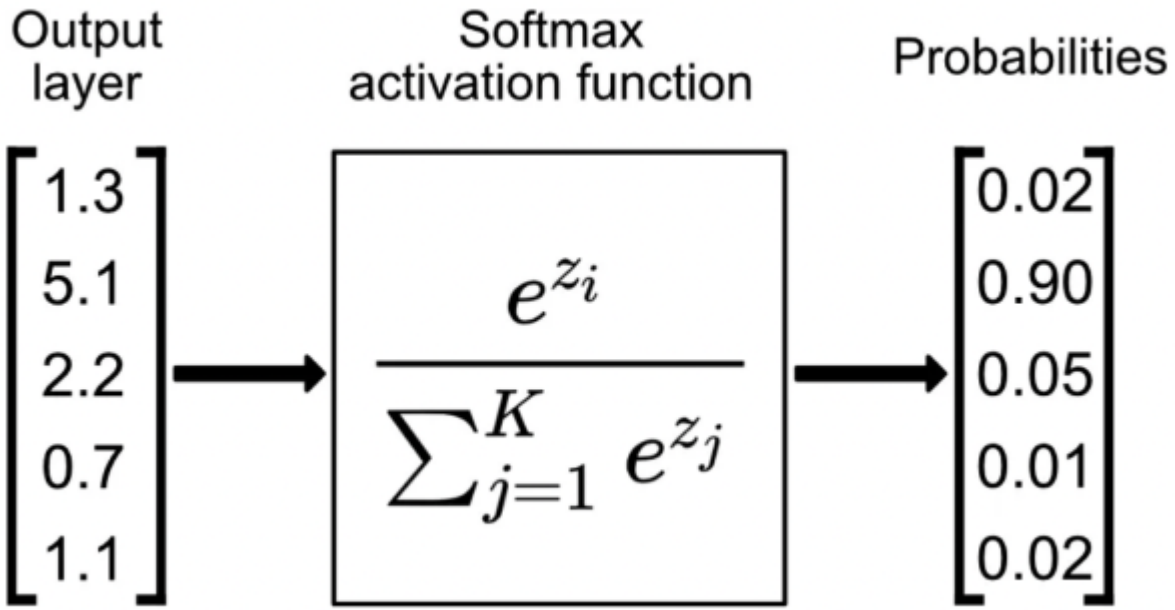
$e^{5.1} = 164.0219$

$e^{2.2} = 9.025013$

$e^{0.7} = 2.0137527$

$e^{1.1} = 3.004166$

$\sum_{j=1}^{k} e^{zj}$=181.734



$e^{1.3} = 3.669297$
$e^{5.1} = 164.0219$
$e^{2.2} = 9.025013$
$e^{0.7} = 2.0137527$
$e^{1.1} = 3.004166$
$\sum_{j=1}^{k} e^{zj}$=181.734

Given

$$z = [1.3,\ 5.1,\ 2.2,\ 0.7,\ 1.1]$$

$$s = [0.02,\ 0.90,\ 0.05,\ 0.01,\ 0.02]$$

$$y = [0,\ 1,\ 0,\ 0,\ 0]$$

For Softmax + Cross-Entropy:

$$\delta \equiv \frac{\partial L}{\partial z} = s - y \qquad \frac{\partial L}{\partial z_j} = s_j - y_j$$

Final gradient (error vector):

$$\boxed{\frac{\partial L}{\partial z} = [\,0.02,\ -0.10,\ 0.05,\ 0.01,\ 0.02\,]}$$

$$\boxed{W^{\text{new}} = W - \eta\,(\delta\, a^{\top})}$$

where:

- δ is a column vector of errors ($K \times 1$)
- $a^{\top}$ is a row vector of inputs ($1 \times n$)
- so $\delta a^{\top}$ produces a $K \times n$ gradient matrix matching W's shape.

SGD update (learning rate η):

$$w_{ji} \leftarrow w_{ji} - \eta\, \delta_j\, a_i, \qquad b_j \leftarrow b_j - \eta\, \delta_j$$

Example

```
[['the', 'earth', 'revolves', 'around', 'sun'], ['the', 'moon', 'revolves', 'around', 'earth']]
['around', 'earth', 'moon', 'revolves', 'sun', 'the']
(6, 10)  W1
(10, 6)  W2
X vector
[0, 0, 0, 0, 0, 1]
----------------
[[0.07499588]
 [0.06991191]
 [0.1069898 ]
 [0.15103953]
 [0.5209983 ]
 [0.07606458]]
----------------
Y-true
[0, 1, 0, 0, 0, 0]
```

Y-predicted=[0,0,0,0,1,0]

- Simplified implementation from https://www.geeksforgeeks.org/implement-your-own-word2vecskip-gram-model-in-python/
- https://colab.research.google.com/drive/1U1PWzRDHsMxwkcdDTglV0p5xibkxggVx?usp=sharing

Pretrained Word2vec

- Download GoogleNews-vectors-negative300.bin.gz
- Developed by Google
- 3 million word embeddings
- 300-dim vectors
- These word vectors are trained on a massive corpus of news articles and web text, enabling them to capture the semantic meaning and relationships between words.

Using Sigmoid Activation function

Note: Self Study content

- The intuition of skip-gram is:

1. Treat the target word and a neighboring context word as positive examples.
2. Randomly sample other words in the lexicon to get negative samples
3. Use logistic regression to train a classifier to distinguish those two cases
4. Use the regression weights as the embeddings

Example

Window size=2

```
... lemon,  a [tablespoon of apricot jam,      a] pinch ...
               c1         c2    t     c3       c4
```

Our goal is to train a classifier such that, given a tuple (t, c) of a target word t paired with a candidate context word c (for example (apricot, jam), or perhaps (apricot, aardvark) it will return the probability that c is a real context word (true for jam, false for aardvark)

positive examples +

t	c
apricot	tablespoon
apricot	of
apricot	preserves
apricot	or

negative examples -

t	c	t	c
apricot	aardvark	apricot	twelve
apricot	puddle	apricot	hello
apricot	where	apricot	dear
apricot	coaxial	apricot	forever

Example

- Our goal is to train a classifier such that, given a tuple (t, c) of a target word t paired with a candidate context word c (for example (apricot, jam), or perhaps (apricot, aardvark) it will return the probability that c is a real context word (true for jam, false for aardvark)

$$P(+|t,c) \tag{6.23}$$

$$P(-|t,c) = 1 - P(+|t,c) \tag{6.24}$$

- How to choose negative context words?

How does the classifier compute the probability P?

- A word is likely to occur near the target if its embedding is similar to the target embedding.
- Two vectors are similar if they have a high dot product (cosine, the most popular similarity metric, is just a normalized dot product).

$$Similarity(t,c) \approx t \cdot c \qquad (6.25)$$

- To turn the dot product into a probability, we'll use the logistic or sigmoid function σ(x), the fundamental core of logistic regression:

$$\sigma(x) = \frac{1}{1+e^{-x}} \tag{6.26}$$

The probability that word c is a real context word for target word t is thus computed as:

$$P(+|t,c) = \frac{1}{1+e^{-t\cdot c}} \tag{6.27}$$

Thank You!

For more than one true context word and target pair

$$P(+|t, c_{1:k}) = \prod_{i=1}^{k} \frac{1}{1 + e^{-t \cdot c_i}} \tag{6.29}$$

$$\log P(+|t, c_{1:k}) = \sum_{i=1}^{k} \log \frac{1}{1 + e^{-t \cdot c_i}} \tag{6.30}$$

Loss function

- That is, we want to maximize the dot product of the word with the actual context words, and minimize the dot products of the word with the k negative sampled non-neighbor words.

$$= \log \frac{1}{1+e^{-c\cdot t}} + \sum_{i=1}^{k} \log \frac{1}{1+e^{n_i \cdot t}} \tag{6.34}$$

For one true context word and target pair